

AN2DL - First Challenge Report

Gradient Gang

Paolo Ginefra, Martina Missana, Lorenzo Pettenuzzo, Matteo Romilio Pasqual

paologinefra, martinamissana1, pettenuzzolorenzo, matteopasqual02

278153, 274235, 273597, 274006

November 14, 2025

1 Introduction

This project addresses the first challenge of the *Artificial Neural Networks and Deep Learning* course (A.Y. 2025–2026), which focuses on the supervised classification of *multivariate time series*. The task consists in predicting a subject’s pain intensity level from a collection of sensor-derived temporal signals. Accurate and automated pain-level estimation plays a crucial role in clinical monitoring, rehabilitation, and continuous patient–state assessment.

1.1 Problem Definition

The dataset is composed of n multivariate time series of fixed temporal length $T = 160$. For each sample $i \in \{1, \dots, n\}$, the raw input is a tensor

$$X_i \in R^{F \times T},$$

where $F = 34$ denotes the total number of feature channels. These channels include:

- **4 survey-derived channels**, obtained from rule-based aggregations estimating subjectively perceived pain;
- **30 biomechanical channels**, representing continuous joint-angle measurements across multiple body parts;
- **3 global features**, constant over time and describing subject-level attributes.

Each time series X_i is associated with a categorical label

$$y_i \in \{0, 1, 2\},$$

corresponding respectively to **No Pain**, **Low Pain**, and **High Pain**. The goal is to learn a function

$$f_\theta : R^{F \times T} \rightarrow \{0, 1, 2\},$$

parameterized by θ , that maximizes classification performance under the evaluation metric described below.

1.2 Objectives

The challenge requires the development of neural architectures *from scratch*, without the use of pre-trained models. Our main objectives are:

1. **Design expressive temporal models.** The architecture must effectively capture both short- and long-range temporal dependencies, as well as inter-feature correlations inherent in physiological time series.
2. **Maximize the macro F1-score.** Model performance is evaluated through the macro-averaged F1-score across the three classes:

$$F1_{\text{macro}} = \frac{1}{3} \sum_{c=0}^2 F1_c,$$

where $F1_c$ is the class-specific F1-score.

3. **Explore multiple architectural paradigms.** We systematically compare feedforward networks, recurrent encoders (LSTM/GRU), convolutional models, and autoencoder-based multi-task architectures in order to identify the most effective design for the given problem.

1.3 Approach

Our approach combines a rigorous preprocessing pipeline, principled dimensionality reduction, and automated hyperparameter optimization. Time series are standardized, augmented with global descriptors, optionally compressed via PCA/POD, and then passed to a modular deep-learning pipeline supporting K-fold cross-validation and model-level windowing. Hyperparameters—architectural, optimization-related, and regularization coefficients—are selected through an Optuna-driven search integrated with PyTorch Lightning and a PostgreSQL-backed study for persistent experiment tracking.

The final model is selected as the configuration achieving the highest mean validation F1-score across folds and is subsequently used to generate predictions for the test set.

2 Problem Analysis

This section presents a detailed examination of the dataset and the underlying learning problem. We characterize the structure and statistical properties of the input signals, identify the primary challenges associated with multivariate temporal modeling, and outline the assumptions that guide our architectural and methodological decisions.

2.1 Dataset Characteristics

The dataset is composed of n multivariate time series $\{X_i\}_{i=1}^n$, each of fixed temporal length $T = 160$ and dimensionality $F = 34$. Formally,

$$X_i \in R^{F \times T}, \quad y_i \in \{0, 1, 2\}.$$

The feature space can be partitioned into three semantically distinct groups:

- **Survey-derived channels (4 features):** Rule-based pain estimators extracted from

simple aggregations. These channels often exhibit coarse temporal variability and may encode subjective or low-resolution information.

- **Biomechanical sensor channels (30 features):** High-frequency continuous measurements of joint kinematics (e.g., neck, elbow, knee). These tend to dominate the temporal structure and contain fine-grained motion patterns.
- **Global static features (3 features):** Subject-specific descriptors that remain constant over time and may correlate with pain response trends.

Label distribution is mildly imbalanced, with the *High Pain* class typically underrepresented. Class imbalance directly impacts training, motivating the use of weighted loss functions and stratified K-fold cross-validation.

2.2 Main Challenges

The problem exhibits several key challenges:

1. **Temporal and Multivariate Complexity.** The model must jointly capture long-range dependencies and cross-channel interactions. Given the dimensionality

$$X_i \in R^{34 \times 160},$$

each sample contains 5,440 values, making the learning problem high-dimensional despite the moderate dataset size.

2. **Heterogeneous Feature Types.** Survey channels, biomechanical signals, and global features have fundamentally different statistical properties. A single model must integrate:

- low-frequency subjective indicators,
- high-frequency sensor signals,
- time-invariant metadata.

3. **Limited Dataset Size.** Deep architectures with many parameters risk overfitting. This necessitates strong regularization, robust cross-validation, careful hyperparameter tuning, and architectural constraints.

4. Class Imbalance. The imbalance across $\{0, 1, 2\}$ labels affects gradient dynamics and may bias predictions toward frequent classes. We mitigate this through inverse-frequency class weights and stratified folds.

5. Absence of Pretrained Models. Unlike domains such as vision or language—where pretrained encoders exist—pain-level estimation from kinematic time series lacks established pretrained backbones. Models must therefore be trained entirely from scratch.

2.3 Initial Assumptions

The design of our approach is based on the following assumptions:

1. **Pain-related patterns manifest in temporal dynamics.** We assume that pain levels are reflected not only in static body positions but primarily in temporal trends, variability, and motion irregularities.
2. **Multiscale information is critical.** Temporal dynamics may appear at different resolutions (slow posture drifts vs. fast joint oscillations). This motivates the exploration of convolutional and multiscale encoders.
3. **Global static features act as priors.** Subject-specific metadata may shift the probability distribution over pain classes and should be integrated with the dynamic representation.
4. **Dimensionality reduction can improve generalization.** PCA/POD projections may capture dominant dynamical modes, reduce redundancy, and stabilize the learning process.
5. **Cross-validation is essential.** Given the dataset size, reliable performance estimates require K-fold validation rather than a single train/validation split.

These considerations guided the development of a modular preprocessing pipeline and a flexible architecture search framework, described in detail in the subsequent sections.

3 Methods

This section describes the methodological framework developed for the challenge. We detail the preprocessing pipeline designed to transform raw time series into structured model inputs, as well as the end-to-end training pipeline responsible for model construction, hyperparameter optimization, and validation. The goal is to provide a precise and reproducible description of all components that contribute to the final learning system.

3.1 Pre-processing

All data preparation procedures were implemented in a dedicated pre-processing module designed to convert the raw multivariate pirate–pain time-series dataset into normalized, feature-rich representations suitable for neural network training. Let the raw dataset consist of

$$X \in R^{N \times F \times T},$$

where N denotes the number of samples, F the number of recorded features, and T the number of temporal observations per sample. The module is fully configurable through YAML files and is centered around a `PreProcessor` class, which encapsulates data loading, feature engineering, normalization, global feature extraction, dimensionality reduction, supervised feature selection, and class-weight computation. The output of the module includes normalized time series, subject-level global features, and class weights for imbalanced learning.

3.1.1 Data Loading

Training and test time series, together with the corresponding training labels, are loaded from CSV input files. An unused trailing column, originating from the acquisition pipeline, is removed before further processing. After loading, each sample is represented as

$$X_i = \{x_i(t)\}_{t=1}^T \in R^{F \times T}.$$

3.1.2 Feature Engineering

Categorical eye-status information is mapped to a binary pirate-indicator variable,

$$\text{isPirate} \in \{0, 1\},$$

and redundant anthropometric variables are discarded. Optional one-hot encoding may be applied. These operations ensure a consistent and compact representation of categorical metadata.

3.1.3 Normalization

All continuous features are standardized using Z-score normalization, computed exclusively on the training set. For each feature f and time index t ,

$$\hat{x}_{i,f}(t) = \frac{x_{i,f}(t) - \mu_f}{\sigma_f},$$

where μ_f and σ_f are respectively the sample mean and standard deviation estimated from the training data. Selected features may be excluded from normalization.

3.1.4 Global Feature Extraction

For each sample X_i , a set of fixed-length global features is computed to summarize temporal dynamics. Three classes of descriptors are extracted:

- **Statistical features:** mean, variance, minimum, maximum, and range,

$$\text{mean}_f = \frac{1}{T} \sum_{t=1}^T x_{i,f}(t),$$

$$\text{var}_f = \frac{1}{T-1} \sum_{t=1}^T (x_{i,f}(t) - \text{mean}_f)^2,$$

$$\min_f = \min_{1 \leq t \leq T} x_{i,f}(t), \quad \max_f = \max_{1 \leq t \leq T} x_{i,f}(t),$$

$$\text{range}_f = \max_f - \min_f.$$

- **Domain-specific features:** aggregated pain-survey descriptors and joint-activity statistics.

The resulting global feature vector for each sample is denoted by

$$g_i \in R^D,$$

where D depends on the selected feature set.

3.1.5 PCA-POD: Dim. Reduction

When enabled, Principal Component Analysis is applied independently to each temporal feature trajectory. For feature f , let

$$x_{i,f} = (x_{i,f}(1), \dots, x_{i,f}(T))^T \in R^T.$$

PCA yields a decomposition

$$x_{i,f} \approx U_f z_{i,f},$$

where U_f contains the leading eigenvectors and $z_{i,f}$ the corresponding Proper Orthogonal Decomposition (POD) coefficients. The number of retained components is the minimum k such that

$$\sum_{j=1}^k \lambda_{f,j} \geq 0.95 \sum_{j=1}^T \lambda_{f,j},$$

with $\lambda_{f,j}$ denoting the eigenvalue magnitudes. All POD coefficients are appended to the global feature vector.

3.1.6 Supervised Feature Selection

The global features are further refined through a multi-stage supervised selection procedure, including:

1. variance thresholding;
2. removal of highly correlated features ($\rho > 0.95$);
3. Random Forest feature-importance ranking;
4. Mutual Information scoring.

Depending on the selection strategy, the final subset is defined as the intersection of the top-ranked features across methods. Mandatory identifiers (`sample_index`, `isPirate`) are preserved.

3.1.7 Class-weight Computation

To address class imbalance in the labels $y \in \{1, \dots, C\}$, class weights are computed using the inverse-frequency scheme,

$$w_c = \frac{N}{C N_c},$$

where N_c denotes the number of samples belonging to class c . These weights are exported for use during model training.

3.1.8 Output Generation

The pipeline produces:

- normalized time series $\hat{X} \in R^{N \times F \times T}$;
- global feature matrices $G \in R^{N \times D'}$, with D' dependent on PCA and feature selection;
- class-weight files for training.

All outputs are written to the designated processed-data directory, ensuring complete reproducibility of the experiments.

3.2 Pipeline

The Pipeline module provides a production-grade orchestration framework for end-to-end deep learning experiments with automated hyperparameter optimization. Its central component, the `FinalPipeline` class, performs architecture selection, K -fold cross-validation, checkpointing, distributed training, and submission generation. The system integrates Optuna (TPE sampler, Median Pruner), PostgreSQL for persistent experiment tracking, PyTorch Lightning for modular training and logging, and TensorBoard for monitoring. The module supports multiple macro-architectures made of modular micro architectures. Given a dataset

$$X \in R^{N \times F \times T}, \quad G \in R^{N \times D},$$

the pipeline automates the selection of model parameters θ , trained via cross-validation to maximize a validation metric (mean F1-score).

3.2.1 Initialization

The pipeline constructs or resumes an Optuna study stored in PostgreSQL, initializes a TPE sampler (seeded) and a Median Pruner, and loads all dataset paths and K-fold parameters:

$$S = \text{OptunaStudy}(\text{TPE}, \text{MedianPruner}).$$

3.2.2 Hyperparameter Optimization

The optimization routine executes n_{trials} Optuna trials. For each trial j , Optuna samples a hyperparameter configuration

$$\theta^{(j)} \sim q(\theta \mid S),$$

where q is the TPE-induced density estimator.

The objective function evaluates the mean F1-score across K folds:

$$\mathcal{L}(\theta^{(j)}) = \frac{1}{K} \sum_{k=1}^K \text{F1}_{\text{val}}^{(k)}(\theta^{(j)}).$$

Each fold involves:

- creation of training/validation splits,
- construction of encoder/decoder/head modules according to $\theta^{(j)}$,
- application of He initialization to all convolutional and linear layers,

$$W \sim \mathcal{N}\left(0, \frac{2}{\text{fan_in}}\right),$$

- optional wrapping in a windowed model,
- PyTorch Lightning training with early stopping and checkpointing.

The trial is pruned when its intermediate value fails to exceed the median of previous trials.

3.2.3 Architecture Construction

The architecture builder assembles a complete model from components:

$$f_{\theta} : (X, G) \mapsto \hat{y},$$

where the encoder may be:

RNN(LSTM/GRU), Conv1D, MultiScaleCNN.

For autoencoder models, a symmetric decoder reconstructs the input:

$$\hat{X} = d_{\theta}(e_{\theta}(X)).$$

The classification head consists of fully connected layers with dropout and non-linearities.

3.2.4 Windowing and Aggregation

If windowing is enabled, the model operates on subsequences

$$X_{i,w} \in R^{F \times T_w},$$

where T_w is the window size and the stride is

$$s = \alpha T_w, \quad \alpha \in [0.25, 1].$$

Aggregation methods include:

$$\text{avg_probs} : \quad \hat{y}_i = \frac{1}{W} \sum_{w=1}^W p_{i,w},$$

$$\text{avg_logits} : \quad z_i = \frac{1}{W} \sum_{w=1}^W z_{i,w}, \quad \hat{y}_i = \text{softmax}(z_i),$$

$$\text{majority_vote} : \quad \hat{y}_i = \text{mode}\{\arg \max p_{i,w}\},$$

$$\text{max_confidence} : \quad \hat{y}_i = \arg \max_c \max_w p_{i,w}(c).$$

3.2.5 Model Selection and Reconstruction

After optimization, the best trial

$$\theta^* = \arg \max_{\theta^{(j)}} \mathcal{L}(\theta^{(j)})$$

is identified. The pipeline loads the corresponding checkpoint, reconstructs the architecture using the stored hyperparameters, applies windowing if necessary, and prepares the model for evaluation.

3.2.6 Submission Generation

For test samples X^{test} , the submission generator computes predictions

$$\hat{y}^{\text{test}} = f_{\theta^*}(X^{\text{test}}, G^{\text{test}}),$$

applies the selected aggregation strategy, maps integer labels to:

$$\{0, 1, 2\} \mapsto \{\text{no_pain}, \text{low_pain}, \text{high_pain}\},$$

and exports a CSV file with zero-padded indices.

3.3 Module Components

3.3.1 DataLoader

The K-fold DataModule creates stratified splits:

$$\bigcup_{k=1}^K \mathcal{D}_{\text{train}}^{(k)} = \mathcal{D}, \quad \mathcal{D}_{\text{train}}^{(k)} \cap \mathcal{D}_{\text{val}}^{(k)} = \emptyset.$$

Batches follow the format:

$$X \in R^{B \times F \times T}, \quad G \in R^{B \times D}.$$

3.3.2 Architectures

Supported:

- RNN encoders: multi-layer LSTM/GRU with optional bidirectionality,
- Conv1D encoders: temporal convolutions with pooling,
- MultiScaleCNN: parallel convolutional branches at multiple receptive fields,
- autoencoder decoders: symmetric to encoders,
- feedforward classification heads.

3.3.3 Submission Generator

Generates prediction matrices

$$P \in R^{N_{\text{test}} \times C},$$

applies aggregation rules, converts class indices to textual labels, and writes the final CSV.

3.3.4 Utilities

Additional utilities include validation of parameter configurations, ensemble model support, feature-selection wrappers, and automatic He initialization.

4 Experiments

This section details the experimental setup adopted to evaluate candidate models. We present the training protocol, cross-validation strategy, and performance metrics, and explain how hyperparameter optimization is conducted. The reported experiments serve as the empirical foundation for assessing the effectiveness of the proposed architectures and preprocessing techniques.

4.1 Experimental Setup

All experiments were conducted using the **FinalPipeline** framework, which integrates preprocessing, model construction, K-fold cross-validation, and Optuna-based hyperparameter optimization into a unified workflow. Unless otherwise specified, each configuration was evaluated under $K = 5$ stratified folds to obtain robust and low-variance performance estimates.

4.1.1 Training Protocol

For every trial generated by the Optuna study, the following steps were executed:

1. Construction of the encoder (LSTM/GRU, Conv1D, or MultiScaleCNN) and, when applicable, a decoder for autoencoder-based models.
2. Application of He initialization to convolutional and linear layers.
3. Training over all K folds using PyTorch Lightning with
4. Model-level windowing and Aggregation of predictions, when enabled.

4.1.2 Evaluation Metrics

Performance is assessed using the macro-averaged F1-score:

$$F1_{\text{macro}} = \frac{1}{3} \sum_{c=0}^2 F1_c,$$

where $F1_c$ denotes the class-specific F1-score computed for class $c \in \{0, 1, 2\}$. This metric is particularly suitable for class-imbalanced multiclass problems such as pain-level prediction.

Additionally, during development we monitor:

- validation loss,
- per-class precision and recall,
- training/validation time per fold,
- number of parameters and computational cost of each model.

5 Results

This section reports the performance achieved by the evaluated models and summarizes the quantitative findings of the study. Comparative tables, figures, and ablations are presented to highlight the strengths and weaknesses of different architectures, preprocessing choices, and hyperparameter configurations.

TODO Present your main findings here. You might want to:

- Compare your results with baselines
- Highlight key achievements using **bold text**
- Explain any unexpected outcomes

6 Discussion

This section provides a critical analysis of the obtained results. We discuss the behavior of the models, interpret key trends, and evaluate the impact of preprocessing, dimensionality reduction, windowing, and architecture choices. Limitations of the current approach and potential sources of error are examined.

TODO In this section, analyse your results critically. Consider:

- Strengths and weaknesses
- Limitations and assumptions

7 Conclusions

This final section summarizes the contributions of the project, reflecting on the methodological and empirical insights obtained throughout the study. We restate the main findings, discuss broader implications, and outline potential directions for future work aimed at further improving time series classification performance.

TODO Summarise your work and discuss potential future directions. This is where you can:

- Restate main contributions
- Suggest improvements
- Propose future work

References